DATA ENGINEERING · PORTFOLIO

안녕하세요,
데이터 엔지니어 안성윤입니다.



SSAFY 15기에서 데이터 파이프라인과 AI 프로젝트를 하고 있습니다. 공개 GitHub 이벤트 6개월치, 6억 6천만 건을 Spark로 집계하는 일을 맡았고, 분석 결과를 다시 쓸 때는 같은 코드인지까지 확인하도록 만들었습니다. 개발을 시작하기 전에는 에스가드에서 1년 2개월 동안 안전관리자로 일했습니다. 같은 기준이라도 현장마다 확인할 게 달라, 서류로 판단하기 어려운 건 직접 가서 확인했습니다.

anjjong777@naver.com **github.com/seongyun-dev** **seongyun-dev.github.io**

---

01 · DATA ENGINEERING

GitJob

GitHub 활동과 채용공고를 비교하는 서비스

팀장 · 우수상 · 2쪽

02 · AI · DATA

RunWay

미래 심박수로 페이스를 제안하는 AI 러닝 서비스

팀원 · 진행 중 · 3쪽

03 · IOT · SYSTEM

R0-KI

자율주행 아파트 순찰 로봇

팀장 · 우수상 · 4쪽

04 · FULL-STACK

할래말래

보드게임 추천 웹 서비스

팀장 · 최우수상 · 5쪽

경력

2024.09 – 2025.11	에스가드 · 안전관리 (정규직) 작업자 안전교육, 보호구 점검, 작업허가서 관리, 협력업체 안전관리와 현장 점검

교육 · 학력

2026 – 현재	SSAFY 15기 · Python 트랙
2017.03 – 2023.02	동명대학교 냉동공조공학과 졸업

수상

2026	GitJob — 우수상 SSAFY 빅데이터 분산 프로젝트
2026	R0-KI — 우수상 SSAFY AIoT 프로젝트 · 공통 프로젝트 본선 진출
2026	할래말래 — 최우수상 SSAFY 웹 프로젝트

대외활동

2023.01	해외봉사단 학생부단장 · 베트남 나트랑 보육원
2022.01 – 2022.12	동명대학교 총동아리연합회 회장

자격증

2026.09.11	정보처리기사
2026.06.19	SQLD (SQL 개발자)
필기 합격	빅데이터분석기사
2023.06.09	산업안전산업기사
2023.04.15	한국사능력검정시험 2급

프로젝트에서 써 본 기술

언어 · 도구	Python · SQL · Docker · Linux · Git · Jira
Data	Spark · PySpark · Kafka · HDFS · Redis · Prometheus · Grafana
AI	PyTorch · GRU · Transformer · W&B
Web · IoT	Django · Vue · ESP32 · Jetson · ROS2 · MQTT

01 · DATA ENGINEERING

GitJob GitHub × Job Matching

2026.08.18 – 10.02 · 6명 · 팀장

github.com/seongyun-dev/GitJob

팀 프로젝트 우수상

개발자의 GitHub 저장소에서 실제로 쓴 기술을 찾아 채용공고의 기술 수요와 비교하는 서비스입니다. 팀장을 맡았고, 팀 파이프라인에서 배치 처리와 HDFS·Spark, GitHub 저장소 매핑을 담당했습니다.

660,081,904건
GH Archive 6개월 이벤트 집계
(시간별 파일 4,416개)

약 580GiB
HDFS에 보관한 원본
(압축 해제 기준)

47,466,125개
활동량을 집계한 저장소
(Spark 1회 약 3시간 10분)

−17.17%
GitHub API 호출
(2,971회 → 2,461회)

GH Archive 원본 6개월 · 4,416개 파일 → **HDFS 보관** 압축 해제 약 580GiB → **Spark 집계** 저장소별·주차별 Parquet → **Artifact 확보·재사용** repo_id + commit SHA → **분석 파트** 기술 근거, 공고 매칭

▢ 제가 맡은 부분 · 실시간 이벤트는 팀이 만든 Kafka consumer를 거쳐 같은 Parquet 형식으로 들어와 주차별 집계에서 합쳐집니다.

맡은 일

- **Spark 집계** — 6억 6천만 건을 분산 집계해 저장소별·주차별 활동량과 Top-N을 Parquet으로 만들었습니다.
- **실시간 경로 연결** — 팀 consumer의 결과를 원본 집계와 같은 형식으로 맞춰 합치고, Snapshot·Checkpoint로 필요한 구간만 다시 처리할 수 있게 했습니다.
- **Artifact 재사용** — 같은 코드 버전일 때만 분석 결과를 다시 쓰고, Redis는 찾는 인덱스로 썼습니다.
- **모니터링** — Prometheus·Grafana로 서버 자원과 Kafka, HDFS 상태를 확인했습니다.

같은 저장소여도, 커밋이 같을 때만 결과를 다시 썼습니다

문제 같은 저장소를 분석할 때마다 GitHub API를 처음부터 다시 호출했습니다.

판단 저장소 단위로 재사용하면 호출은 더 줄지만, 커밋이 바뀌면 예전 코드의 결과가 나갑니다.

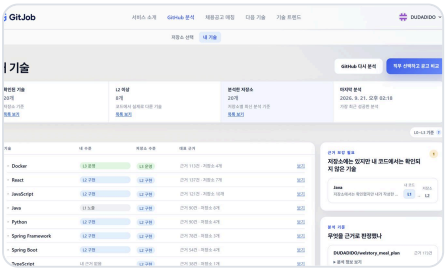
결과 repo_id + commit SHA가 같을 때만 재사용해 API 호출 17.17% 감소. 처리 시간(−16.71%)은 외부 요인이 섞인 참고 수치로만 봅니다.

계획만 봤는데, 다음에 본 계획이 달라져 있었습니다

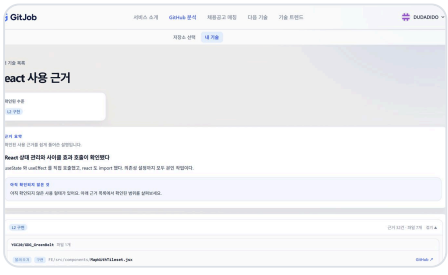
문제 backfill 도구를 dry run(계획만 보는 모드)으로 다시 돌리면 결과가 바뀌었습니다.

원인 재개용 checkpoint 기록이 dry run 확인보다 먼저 실행되고 있었습니다.

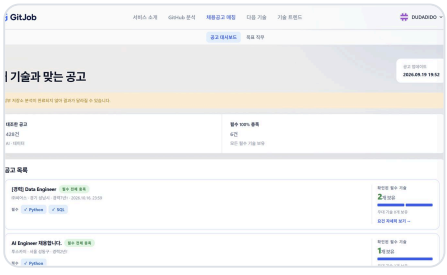
결과 실제 실행에서만 기록하도록 고치고, 반복 dry run이 바이트 단위로 같은지 등 테스트 6개를 추가했습니다.



내 기술: 저장소에서 확인한 기술과 수준



기술별 사용 근거 (코드 위치)



내 기술과 맞는 채용공고

Python · PySpark · Kafka · HDFS · Parquet · Redis · Docker · Prometheus · Grafana

02 · AI · DATA

RunWay AI Running Pacemaker

2026.10.06 – 11.18 · 6명 · 팀원
AI 파트
진행 중

현재 심박수와 페이스, 코스 경사로 미래 심박수를 예측해 적정 페이스와 러닝 코스를 제안하는 서비스입니다. AI 파트에서 학습 데이터 전처리·검증, 심박 예측 모델, 코스 생성 로직을 맡았습니다.

약 69,169개
전처리한 러닝 세션
(FitRec)

295,746 → 0건
경사 표시 규칙에
어긋난 기록 정리

70.9%↓
5km 코스 그래프 생성
(2.13초 → 0.62초)

12.0% → 0.6%
거꾸로 된 심박 예측
(MAE 3.143 → 3.159 bpm)

맡은 일

- **학습 데이터 검증** — 사용자 단위로 학습·검증·시험 데이터를 나눠 같은 사람의 기록이 평가에 섞이지 않게 했고, GPS·경사·끊긴 시간 구간을 점검했습니다.
- **모델 비교** — 심박이 늦게 반응하는 점을 반영해 ODE 모델부터 GRU, Transformer까지 비교했습니다.
- **공통 평가 기준** — 팀원들의 LightGBM, GRU, Transformer를 같은 데이터와 같은 평가 코드로 비교하게 했습니다.
- **코스 데이터** — 부산 OSM 도로 73,064 ways를 코스 생성용 그래프로 만들고, 국내 러닝 GPX 299개를 모았습니다.

느린 건 길찾기가 아니라 그래프 만드는 부분이었습니다

문제 코스 추천 첫 요청이 5km 3.72초, 10km 12.92초 걸렸습니다.

원인 단계별로 확인해 보니 그래프 생성이 가장 오래 걸렸고, add_edge 호출만 약 37만 번이었습니다.

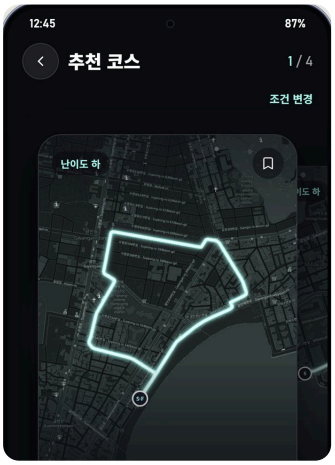
결과 지도를 격자로 나눠 요청 범위의 길만 넣어 그래프 생성 70.9% 단축. 기존 방식과 10개 조건에서 경로·순위·추천 TOP3가 같은지 확인했습니다.

평균은 가장 정확했지만, 가끔 거꾸로 예측했습니다

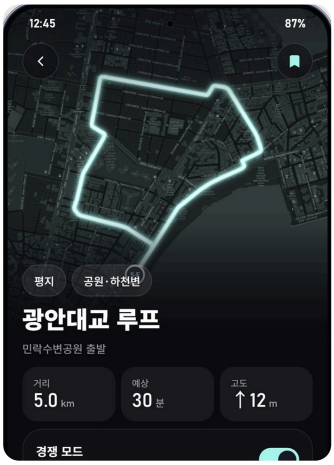
문제 MAE가 가장 작은 모델(3.143 bpm)도 속도를 초속 0.5m 올리면 12.0%에서 심박이 내려간다고 예측했습니다.

원인 평균 오차는 예측 방향을 보여 주지 않습니다. 페이스 제안에 쓰면 맞지 않는 페이스가 나올 수 있습니다.

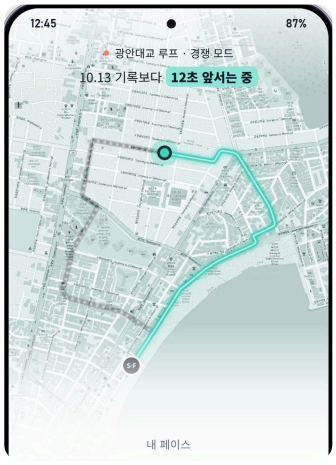
결과 Transformer 1개와 GRU 3개를 합친 모델을 방향까지 맞추도록 추가 학습해 MAE 3.159 bpm, 거꾸로 된 예측 0.6%.



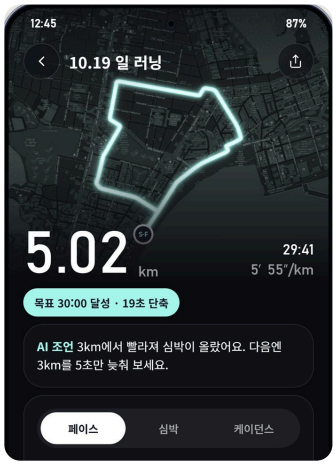
추천 코스 목록



코스 상세



달리는 중



기록 상세

구현 전 화면 설계 v9 · 지도 © OpenStreetMap contributors

Python · PyTorch · GRU · Transformer · W&B · NetworkX · OpenStreetMap

03 · IOT · SYSTEM INTEGRATION

RO-KI Apartment Patrol Robot

2026.07.06 – 08.14 · 7명 · 팀장

github.com/seongyun-dev/R0-KI

우수상 · 본선 진출

아파트 단지를 순찰하며 이상 상황을 감지하고, 로봇 상태를 실시간으로 보여 주는 자율주행 순찰 로봇입니다. 팀장이자 IoT 파트를 맡아 센서와 로봇, 서버 사이 통신을 연결했습니다.

User Command 사용자 명령 → **Backend → MQTT** 메시지 전달 → **ROS2 Topic** 로봇 쪽 수신 → **Robot State** 센서 값 · 로봇 상태 → **Robot Action** 실제 동작

▢ 제가 맡은 부분 · 구간마다 다른 형태로 나오던 로그를 한 형식으로 맞춰, 명령이 어느 단계에서 멈췄는지 따라갈 수 있게 했습니다.

맡은 일

- **통신 연동** — ESP32, Jetson Orin Nano, ROS2, MQTT로 센서와 로봇, 서버 사이 통신을 연결했습니다.
- **로그 파싱·정규화** — MQTT 메시지, ROS2 Topic, 센서 값, 로봇 상태처럼 장치마다 다른 로그를 모니터링 화면이 쓸 수 있는 한 형식으로 맞췄습니다.
- **로봇 상태 데이터** — 배터리·위치·부위별 상태 값을 정리해 로봇 목록과 상세 화면에 들어가게 했습니다. 화면 자체는 팀원이 만들었습니다.
- **지도 화면 보정** — 2.5D 지도와 순찰 구역 편집 화면을 다듬었습니다.
- **팀장** — Jira로 파트별 진행 상황과 통합 일정을 맞췄습니다.

명령이 어디서 멈췄는지, 로그로 따라갔습니다

문제 여러 시스템을 연결하자 로봇이 예상대로 움직이지 않을 때가 있었습니다.

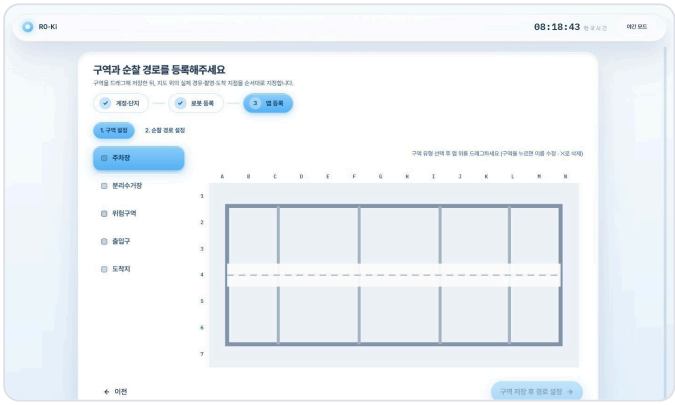
원인 명령이 Backend, MQTT, ROS2를 차례로 거치는데 시스템마다 로그 형태가 달라, 어느 단계에서 멈췄는지 바로 알 수 없었습니다.

해결 로그를 파싱하고 정규화해 한곳에서 볼 수 있게 했습니다.

결과 사용자 명령부터 로봇 동작까지 단계별로 따라가며 멈춘 단계를 찾아 고쳤습니다.



로봇 상세: 정리한 상태 데이터가 표시되는 화면 (화면은 팀원 구현)



순찰 구역 등록

ESP32 · Jetson Orin Nano · ROS2 · MQTT · Jira

04 · FULL-STACK

할래말래 Board Game Recommender

2026.06.04 – 06.19 · 2명 · 팀장

SSAFY 웹 프로젝트

팀 프로젝트 최우수상

인원·시간·난이도·장르 조건에 맞는 보드게임을 추천하고, 게임 정보와 커뮤니티, 매장 지도를 보여 주는 웹 서비스입니다. 2인 팀의 팀장으로 기획과 일정을 맡았고, 기능 단위로 나눠 화면부터 API까지 각자 만들었습니다.

맡은 일

- **커뮤니티** — 말머리별 게시판의 Vue 화면과 Django API
- **마이페이지** — 화면과 API
- **보드게임 랭킹** — 인기 게임 랭킹 화면과 API
- **지도 연동** — 보드게임 매장 정보를 지도에 표시
- **팀장** — 서비스 기획과 일정 조율

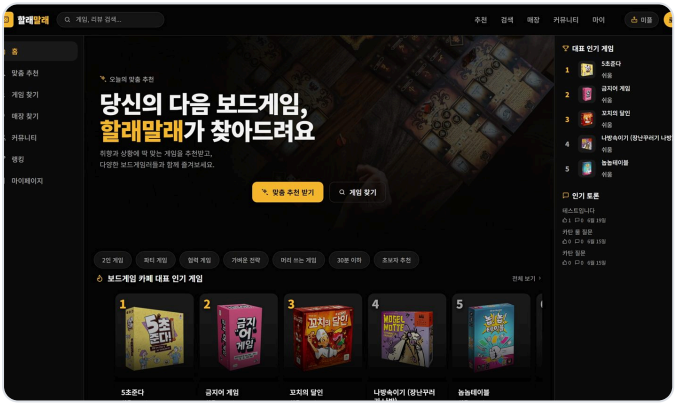
둘이니까 말로 맞추면 될 줄 알았습니다

문제 기능 요구사항을 충분히 정하지 않고 시작했더니, 서로 생각한 필수 기능이 달라 만든 부분을 여러 번 고쳐야 했습니다.

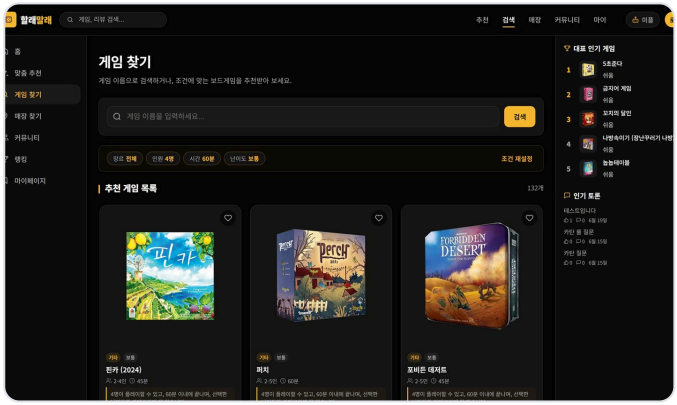
시도 기능마다 누구 의견이 나은지 따져 정했지만, 다음 기능에서 같은 논의가 반복됐습니다.

해결 사용자 흐름을 다시 그려 필수·추가 기능을 나누고, 필수 기능부터 끝내기로 합의했습니다.

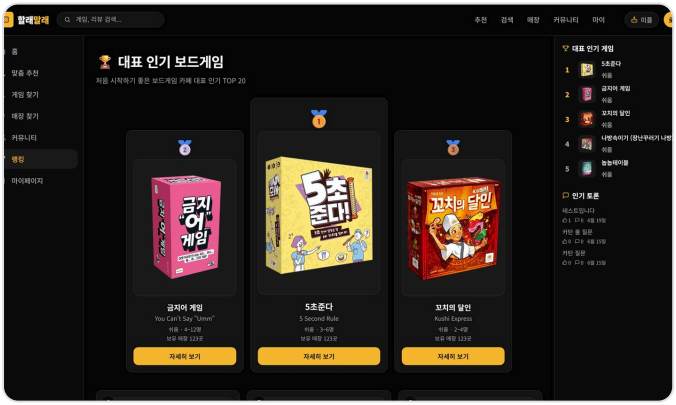
결과 새 아이디어는 그 흐름에 필요한지부터 확인했고, 서비스를 완성해 최우수상을 받았습니다.



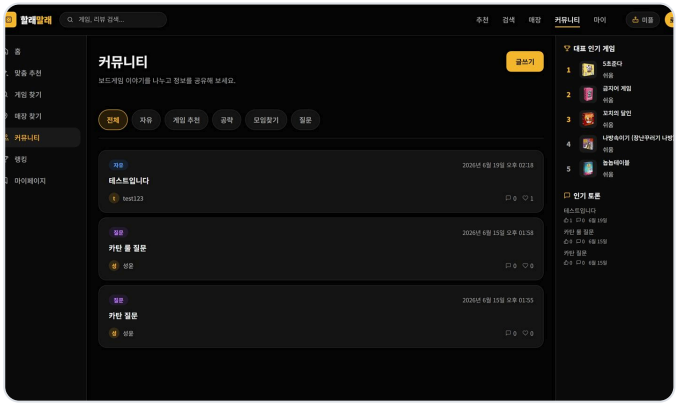
홈: 오늘의 맞춤 추천과 인기 보드게임



조건으로 받은 추천 결과



보드게임 랭킹 (제가 만든 화면)



커뮤니티 (제가 만든 화면)

Django · Django REST Framework · Vue 3 · SQLite · JWT